Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Airtificial Intellegence Practical No :-VII

Roll No :- T053	**Name :- Nidhi Yadav**
Class :- TYIT	**Batch :- 02**
Date of Assignment :- 1-08-2026	**Date/Time of Submission :- 1-08-2026**

Machine Learning

1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.

Import Libraries

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
```

✓ 0.0s

Load Iris Dataset & Convert into Dataframe

```
iris = load_iris()
```

✓ 0.0s

```
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["Target"] = iris.target
```

✓ 0.0s

Display First Five Row

```
print("Dataset:")
print(df.head())
```

✓ 0.0s

```
Dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   Target
0       0
1       0
2       0
3       0
4       0
```

Check Missing Values

```
print("\nMissing Values:")
print(df.isnull().sum())
```

✓ 0.0s

```
Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
Target               0
dtype: int64
```

Features and Target

```
X = df.iloc[:, :-1]
y = df["Target"]
```

✓ 0.0s

Split Data into Training and Testing

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)
```

✓ 0.0s

Train Decision Tree Model

```
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
```

✓ 0.0s

Tree Model Output :-

DecisionTreeClassifier

▸ Parameters

▸ Fitted attributes

Display Accuracy

```
print("\nTraining Accuracy:", model.score(X_train, y_train))
print("Testing Accuracy:", model.score(X_test, y_test))
print("Nidhi  Yadav  T053")
```

✓ 0.0s

```
Training Accuracy: 1.0
Testing Accuracy: 0.9666666666666667
Nidhi  Yadav  T053
```

Dataset :- zomato.csv

Import pandas

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScaler
```

✓ 0.0s

Load Dataset

```
df = pd.read_csv("zomato.csv")
```

✓ 3.4s

Display 5 Rows

```
print("First 5 Rows")
print(df.head())
```

✓ 0.0s

Output:-

```
First 5 Rows
                                                 url  \
0  https://www.zomato.com/bangalore/jalsa-banasha...
1  https://www.zomato.com/bangalore/spice-elephan...
2  https://www.zomato.com/SanchurroBangalore?cont...
3  https://www.zomato.com/bangalore/addhuri-udupi...
4  https://www.zomato.com/bangalore/grand-village...

                                             address                name  \
0  942, 21st Main Road, 2nd Stage, Banashankari, ...               Jalsa
1  2nd Floor, 80 Feet Road, Near Big Bazaar, 6th ...      Spice Elephant
2  1112, Next to KIMS Medical College, 17th Cross...     San Churro Cafe
3  1st Floor, Annakuteera, 3rd Stage, Banashankar...  Addhuri Udupi Bhojana
4  10, 3rd Floor, Lakshmi Associates, Gandhi Baza...         Grand Village

  online_order book_table   rate  votes                     phone  \
0          Yes        Yes  4.1/5    775    080 42297555\r\n+91 9743772233
1          Yes         No  4.1/5    787                 080 41714161
2          Yes         No  3.8/5    918                +91 9663487993
3           No         No  3.7/5     88                +91 9620009302
4           No         No  3.8/5    166  +91 8026612447\r\n+91 9901210005

       location          rest_type  \
0  Banashankari      Casual Dining
1  Banashankari      Casual Dining
...
1        Buffet    Banashankari
2        Buffet    Banashankari
3        Buffet    Banashankari
4        Buffet    Banashankari
```

Display Dataset Information

```
print("\nDataset Information")
print(df.info())
```

✓ 0.0s

Output :-

```
Dataset Information
<class 'pandas.DataFrame'>
RangeIndex: 51717 entries, 0 to 51716
Data columns (total 17 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   url                          51717 non-null  str
 1   address                      51717 non-null  str
 2   name                         51717 non-null  str
 3   online_order                 51717 non-null  str
 4   book_table                   51717 non-null  str
 5   rate                         43942 non-null  str
 6   votes                        51717 non-null  int64
 7   phone                        50509 non-null  str
 8   location                     51696 non-null  str
 9   rest_type                    51490 non-null  str
 10  dish_liked                   23639 non-null  str
 11  cuisines                     51672 non-null  str
 12  approx_cost(for two people)  51371 non-null  str
 13  reviews_list                 51717 non-null  str
 14  menu_item                    51717 non-null  str
 15  listed_in(type)              51717 non-null  str
 16  listed_in(city)              51717 non-null  str
dtypes: int64(1), str(16)
memory usage: 6.7 MB
None
```

Shape:-

```
print("\nDataset Shape")
print(df.shape)
```

✓ 0.0s

```
Dataset Shape
(51717, 17)
```

Column :-

```
print("\nColumn Names")
print(df.columns)
```

✓ 0.0s

```
Column Names
Index(['url', 'address', 'name', 'online_order', 'book_table', 'rate', 'votes',
       'phone', 'location', 'rest_type', 'dish_liked', 'cuisines',
       'approx_cost(for two people)', 'reviews_list', 'menu_item',
       'listed_in(type)', 'listed_in(city)'],
      dtype='str')
```

Summary Statistics

```
print("\nSummary Statistics")
print(df.describe())
```

✓ 0.0s

```
Summary Statistics
              votes
count  51717.000000
mean     283.697527
std      803.838853
min        0.000000
25%        7.000000
50%       41.000000
75%      198.000000
max    16832.000000
```

Data Types:-

```
print("\nData Types")
print(df.dtypes)
```

✓ 0.0s

```
Data Types
url                                  str
address                              str
name                                 str
online_order                         str
book_table                           str
rate                                 str
votes                              int64
phone                                str
location                             str
rest_type                            str
dish_liked                           str
cuisines                             str
approx_cost(for two people)          str
reviews_list                         str
menu_item                            str
listed_in(type)                      str
listed_in(city)                      str
dtype: object
```

Check Missing Values :-

```
print("\nMissing Values")
print(df.isnull().sum())
```

✓ 0.0s

```
Missing Values
url                                  0
address                              0
name                                 0
online_order                         0
book_table                           0
rate                              7775
votes                                0
phone                             1208
location                            21
rest_type                          227
dish_liked                       28078
cuisines                            45
approx_cost(for two people)        346
reviews_list                         0
menu_item                            0
listed_in(type)                      0
listed_in(city)                      0
dtype: int64
```

Handle Missing Value :-

```
df["rate"] = df["rate"].fillna("0")
df["dish_liked"] = df["dish_liked"].fillna("Not Available")
df["cuisines"] = df["cuisines"].fillna("Unknown")
```

✓ 0.0s

Remove Duplicate Records:-

```
df = df.dropna()
print("\nMissing Values After Cleaning")
print(df.isnull().sum())
```

✓ 0.0s

```
Missing Values After Cleaning
url                                  0
address                              0
name                                 0
online_order                         0
book_table                           0
rate                                 0
votes                                0
phone                                0
location                             0
rest_type                            0
dish_liked                           0
cuisines                             0
approx_cost(for two people)          0
reviews_list                         0
menu_item                            0
listed_in(type)                      0
listed_in(city)                      0
dtype: int64
```

Label Encoding:-

```
df = df.drop_duplicates()

print("\nShape After Removing Duplicates")
print(df.shape)
```

✓ 1.3s

```
Shape After Removing Duplicates
(50295, 17)
```

Rename Column:-

```
df.rename(columns={
    "approx_cost(for two people)":"cost_for_two"
}, inplace=True)

print("\nColumns After Renaming")
print(df.columns)
```

✓ 0.0s

```
Columns After Renaming
Index(['url', 'address', 'name', 'online_order', 'book_table', 'rate', 'votes',
       'phone', 'location', 'rest_type', 'dish_liked', 'cuisines',
       'cost_for_two', 'reviews_list', 'menu_item', 'listed_in(type)',
       'listed_in(city)'],
      dtype='str')
```

Drop Unnecessary Columns:-

```
df = df.drop(["url","address","phone","reviews_list","menu_item"], axis=1)

# 14. Feature Selection
features = ["votes","cost_for_two","online_order","book_table"]
print("\nSelected Features")
print(features)

# 15. Feature Engineering
df["Restaurant_Age"] = 1
```

✓ 0.1s

```
Selected Features
['votes', 'cost_for_two', 'online_order', 'book_table']
```

Convert Data Type:-

```
df["cost_for_two"] = df["cost_for_two"].astype(str)
df["cost_for_two"] = df["cost_for_two"].str.replace(",", "")
df["cost_for_two"] = pd.to_numeric(df["cost_for_two"], errors="coerce")

df["votes"] = pd.to_numeric(df["votes"], errors="coerce")

df = df.dropna()

print("\nData Types After Conversion")
print(df.dtypes)
```

✓ 0.0s

Remove Null Values:-

```
minmax = MinMaxScaler()

df[["votes","cost_for_two"]] = minmax.fit_transform(
    df[["votes","cost_for_two"]]
)

print("\nData After Min-Max Scaling")
print(df[["votes","cost_for_two"]].head())

# 18. Standard Scaling

scaler = StandardScaler()

df[["votes","cost_for_two"]] = scaler.fit_transform(
    df[["votes","cost_for_two"]]
)

print("\nData After Standard Scaling")
print(df[["votes","cost_for_two"]].head())
```

✓ 0.0s

Output:-

```
Data Types After Conversion
name                 str
online_order       int64
book_table         int64
rate                 str
votes              int64
location           int64
rest_type            str
dish_liked           str
cuisines             str
cost_for_two       int64
listed_in(type)    int64
listed_in(city)      str
Restaurant_Age     int64
dtype: object
```

```
Data After Min-Max Scaling
      votes  cost_for_two
0  0.046043      0.127517
1  0.046756      0.127517
2  0.054539      0.127517
3  0.005228      0.043624
4  0.009862      0.093960

Data After Standard Scaling
      votes  cost_for_two
0  0.601517      0.548574
1  0.616303      0.548574
2  0.777720      0.548574
3 -0.245000     -0.585483
4 -0.148889      0.094951
```

Standard Scaling :-

```
X = df[["votes","cost_for_two","book_table"]]

y = df["online_order"]

print("\nFeatures")
print(X.head())

print("\nTarget")
print(y.head())
print("Nidhi Yadav T053")
```

0.0s

Output:-

```
Features
      votes  cost_for_two  book_table
0  0.601517      0.548574           1
1  0.616303      0.548574           0
2  0.777720      0.548574           0
3 -0.245000     -0.585483           0
4 -0.148889      0.094951           0

Target
0    1
1    1
2    1
3    0
4    0
Name: online_order, dtype: int64
Nidhi Yadav T053
```

Comparison :-

Feature	Iris Dataset	Zomato Dataset
Dataset Type	Flower dataset	Restaurant business dataset
Number of Records	150	51,717
Number of Columns	5	17
Data Type	Mostly numerical	Numerical + Categorical
Target Column	Variety	Online Order (or any selected target)
Missing Values	Very few / None	Many missing values
Preprocessing Required	Basic	More preprocessing needed
Real-life Use	Plant classification	Restaurant analysis and prediction
Complexity	Simple	More complex

Justification:-

- **Iris Dataset is a small and clean dataset. It contains only 150 records and 5 columns, so it is easy to understand and is mainly used for learning basic machine learning concepts.**
- **Zomato Dataset is a real-world business dataset with 51,717 records and 17 columns. It contains both numerical and categorical data, missing values, duplicate records, and text columns. Therefore, it requires more preprocessing such as handling missing values, encoding categorical variables, scaling, and feature selection.**
- **The Iris dataset is suitable for beginners, whereas the Zomato dataset is better for performing real-life data preprocessing and machine learning tasks because it reflects actual business data.**

Conclusion :-

The Iris dataset is useful for understanding the basic machine learning workflow, while the Zomato dataset is more suitable for real-world business analysis because it is larger, more complex, and requires complete data preprocessing before building a machine learning model.